

ee2004

week6

Data Types in C and C++ Languages

- In general C programming textbooks, we see *char*, *short*, *int*, *long*, *float*, and *double* data types. We need to examine the size of C data types in the light of 32-bit processors such as Arm.
- The C standards do not specify the size of data types. The compiler designers are free to decide the size for each data type. The *float* and *double* data types are standardized by the IEEE754.
- The sizes of char and short are often set at 1 byte and 2 bytes.
- The size of int is often depending on the data size of the CPU but rarely go below 16 or above 32.
- The sizes of long and long long are implemented the same way everywhere.
- Use pseudo function sizeof(): C compilers supports a pseudo function sizeof(), which returns the size of the parameter in the number of byte(s). The parameter may be a data type or a variable. For example, sizeof(int) returns the number of bytes in an int variable.

Data type Size Range

- **char** 1 byte -128 to 127
- **unsigned char** 1 byte 0 to 255
- **short int** 2 bytes -32,768 to 32,767
- **unsigned short int** 2 bytes 0 to 65,535
- **int** 4 bytes -2,147,483,648 to 2,147,483,647
- **unsigned int** 4 bytes 0 to 4,294,967,295
- **long** 4 bytes -2,147,483,648 to 2,147,483,647
- **unsigned long** 4 bytes 0 to 4,294,967,295
- **long long** 8 bytes -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
- **unsigned long long** 8 bytes 0 to 18,446,744,073,709,551,615

These integer data types are defined in a header file called *stdint.h*. You need to include this header file in order to use these data types

Data type	Size	Range
int8_t	1 byte	-128 to 127
uint8_t	1 byte	0 to 255
int16_t	2 bytes	-32,768 to 32,767
uint16_t	2 bytes	0 to 65,535
int32_t	4 bytes	-2,147,483,648 to 2,147,483,647
uint32_t	4 bytes	0 to 4,294,967,295
int64_t	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
uint64_t	8 bytes	0 to 18,446,744,073,709,551,615

Bit-wise Operations in C

- One of the most important and powerful features of the C language is its ability to perform bit manipulations.
- While every C programmer is familiar with the logical operators AND (&&), OR (||), and NOT (!),
- many C programmers are less familiar with the bitwise operators AND (&), OR (|), EX-OR (^), invert (~), right shift (>>), and left shift (<<).
- These bit-wise operators are widely used in software engineering for embedded systems and control; consequently, their understanding and mastery are critical in microprocessor-based system design and interfacing.
- 0x35 & 0x0F results in 0x05 /* ANDing */
- 0x04 | 0x68 results in 0x6C /* ORing: */
- 0x54 ^ 0x78 results in 0x2C /* XORing */
- ~0x55 results in 0xAA /* Inverting 0x55 */

Setting and Clearing (masking) bits

- OR can be used to set a bit or bits, and AND can be used to clear a bit or bits.
- Anything ‘OR’ed with a 1 results in a 1;
- anything ‘OR’ed with a 0 results in no change.
- Anything ‘AND’ed with a 1 results in no change;
- anything ‘AND’ed with a 0 results in a zero.
- Anything ‘EX-OR’ed with a 1 results in the complement;
- anything ‘EX-OR’ed with a 0 results in no change.

```c
#include <stdio.h>

int main(void) {
volatile unsigned char temp; /* declare volatile otherwise the optimizer will
remove it. */
temp = 0x35 & 0x0F; /* ANDing : 0x35 & 0x0F = 0x05 */
temp = 0x04 | 0x68; /* ORing : 0x04 | 0x68 = 0x6C */
temp = 0x54 ^ 0x78; /* XORing : 0x54 ^ 0x78 = 0x2C */
temp = ~0x55; /*Inverting : ~0x55 = 0xAA */
while (1);
return 0;
}
```

```c
int main(void)
{
unsigned char var1;

while(1)
{
var1 = var1 | 0x10; /* Set bit 4 (5th bit) of var1 */
var1 = var1 & 0xEF; /* Clear bit 4 (5th bit) of var1 */
var1 = var1 ^ 0x10; /* Togggle bit 4 (5th bit) of var1 */
}
}
```

Testing bit with bit-wise operators in C

- In many cases of system programming and hardware interfacing, it is necessary to test a given bit to see if it is high or low.
- For example, many devices send a high signal to signify that they are ready for an action or to indicate that they have data available.
- How can the bit (or bits) be tested?
- In such cases the unused bits are masked and then the remaining data is tested.
- EX: Write a C program to monitor bit 5 of var1. If it is HIGH, change value of var2 to 0x55; otherwise, change value of var2 to 0xAA.

```
while(1)
{
if (var1 & 0x20) /* check bit 5 (6th bit) of var1 */
var2 = 0x55; /* this statement is executed if bit 5 is a 1 */
else
var2 = 0xAA; /* this statement is executed if bit 5 is a 0 */
}
```

Bit-wise shift operation in C

- >> data >> number of bit-positions to be shifted right
- << data << number of bit-positions to be shifted left

The following shows some examples of shift operators in C:

1. 0b00010000 >> 3 /* it equals 00000010. Shifting right 3 times */
2. 0b00010000 << 3 /* it equals 10000000. Shifting left 3 times */
3. 1 << 3 /* it equals 00001000. Shifting left 3 times */

Compound Operators

- In C language, whenever the left-hand-side of the assignment operator (=) and the first operand on the right-hand-side are identical we can avoid repeating the operand by using the compound operators.

a = a + 6;	a += 6;
a = a – 23;	a –= 23;
y = y * z;	y *= z;
z = z / 25;	z /= 25;
w = w | 0x20;	w |= 0x20;
v = v & mask;	v &= mask;
m = m ^ togBits;	m ^= togBits;

Review Questions

1. What is result of 0x2F & 0x27?
2. What is result of 0x2F | 0x27?
3. What is result of 0x2F ^ 0x27?
4. What is result of 0x2F >> 3?
5. What is result of 0x27 << 4?

Bit-wise operations using compound operators

- The majority of hardware access level code involves
  - setting a bit or bits in a register,
  - clearing a bit or bits in a register,
  - toggling a bit or bits in a register,
  - monitoring the status bits.
- For the first three cases, the operations read the content of the register, modify a bit of bits then write it back to the same register.
- The compound operators are very suitable for these operations.

- To set bit(s) in a register,
- register |= MASK; //where MASK is a number that has ‘1’ at the bit(s) to be set.
- register |= 0x08; //The number 0x08 has a ‘1’ at bit 3, therefore the statement sets bit 3 of the register.
- register |= 0x42; // The number 0x42 has a ‘1’ at bit 6 and bit 1, therefore the statement sets bit 6 and bit 1 of the register.

- To clear bit(s) in a register,
- register &= ~MASK; //where MASK is a number that has ‘1’ at the bit(s) to be cleared.
- register &= ~0x20; //The number 0x20 has a ‘1’ at bit 5, therefore the statement clears bit 5 of the register.
- register &= ~0x12; //The number 0x12 has a ‘1’ at bit 4 and bit 1, therefore the statement clears bit 4 and bit 1 of the register.

Notice the mask for clearing the bits is the same as the mask for setting the bits, where the bits to be modified are ‘1’ and the rest of the bits are ‘0’ except that in clearing the bits, the mask is complemented in the statements.

Using shift operator to generate mask

- With the statements above, one challenge may be to generate the mask with the correct bit(s) set to 1 depending on how proficient you are with converting binary numbers to hexadecimal.
- Some compilers allow you to write a literal binary number in the format of 0b00000000 but since it is not in the C standards, many compilers do not accept this notation.
- One way to ease the generation of the mask is to use the left shift operator.
- To generate a mask with bit n set to 1, use the expression: 1 << n
- If more bits are to be set in the mask, they can be “or” together. To generate a mask with bit n and bit m set to 1, use the expression: (1 << n) | (1 << m)

- Now to set bit 3 of the register, we can rewrite the statement

  register |= 0x08; as register |= 1 << 3;
- And to set bit 6 and bit 1 of the register, we can rewrite the statement

  register |= 0x42; as register |= (1 << 6) | (1 << 1);
- The same goes for clearing bit 5 of the register, we can use the statement

  register &= ~(1 << 5);
- And to clear bit 4 and bit 1 of the register, we can use the statement

  register &= ~((1 << 4) | (1 << 1));

Notice that regardless of setting or clearing bits, the mask always has 1s at the bit locations for the bits to be modified and when multiple bits are used in the mask, they are always 'OR'ed together.

STM Arm I/O Programming;
Using STM32CubeMX to Modify Hardware of the Microcontroller

STM32CubeMX (available under STM32CubeIDE) can be used to control and modify hardware of the STM32F4 microcontroller.

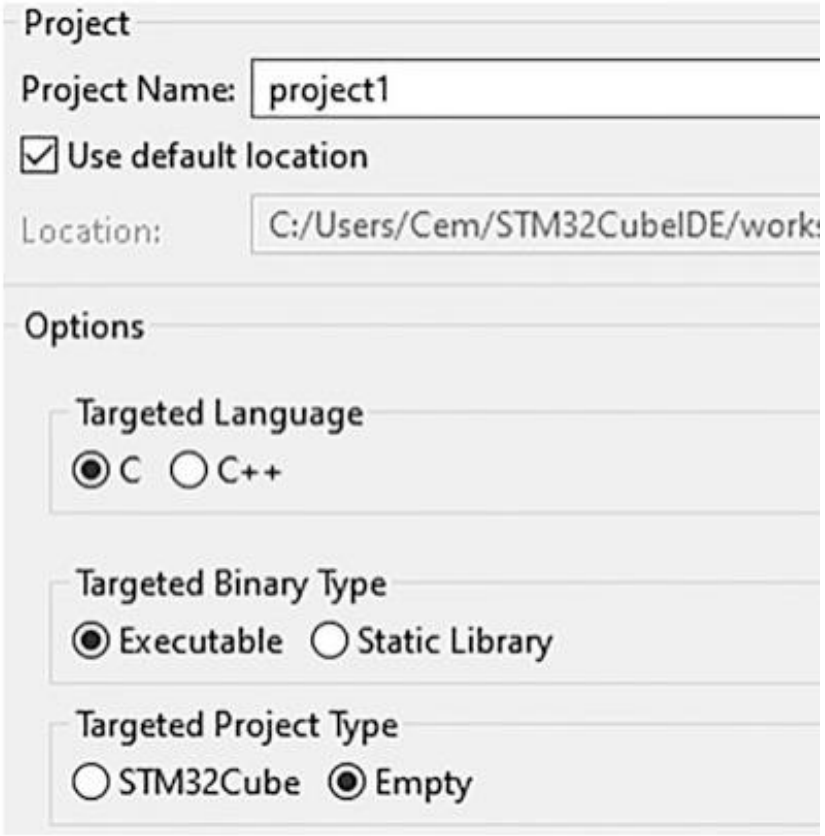
To note here, hardware properties can also be set via available libraries such as hardware abstraction layer (HAL).

The advantage of STM32CubeMX is that operations are done visually on it.

Moreover, it produces a template C code containing predefined functions to be used to control hardware properties.

Creating a New Project Using STM32CubeMX

- In order to modify hardware of the microcontroller, we can create a new project as explained in Sect. 3.1.3 with one difference.
- Since we will modify hardware of the microcontroller, we will set the “Targeted Project Type” as “STM32Cube” in Fig. 3.2.
- Afterward, a new window appears asking for the “Firmware Library Package Setup.” At this step, leave all default settings as they are and press “Finish.”
- The next pop-up window asks whether to initialize all peripheral units with their default mode. Click “No” since we will modify necessary pin properties as we need them.



- A window should open up as in Fig. 3.7.
- We can set all system hardware properties through STM32CubeMX.
- At this step, let's assume that we want to turn on the green LED on the STM32F4 board whenever the user button on it is pressed.

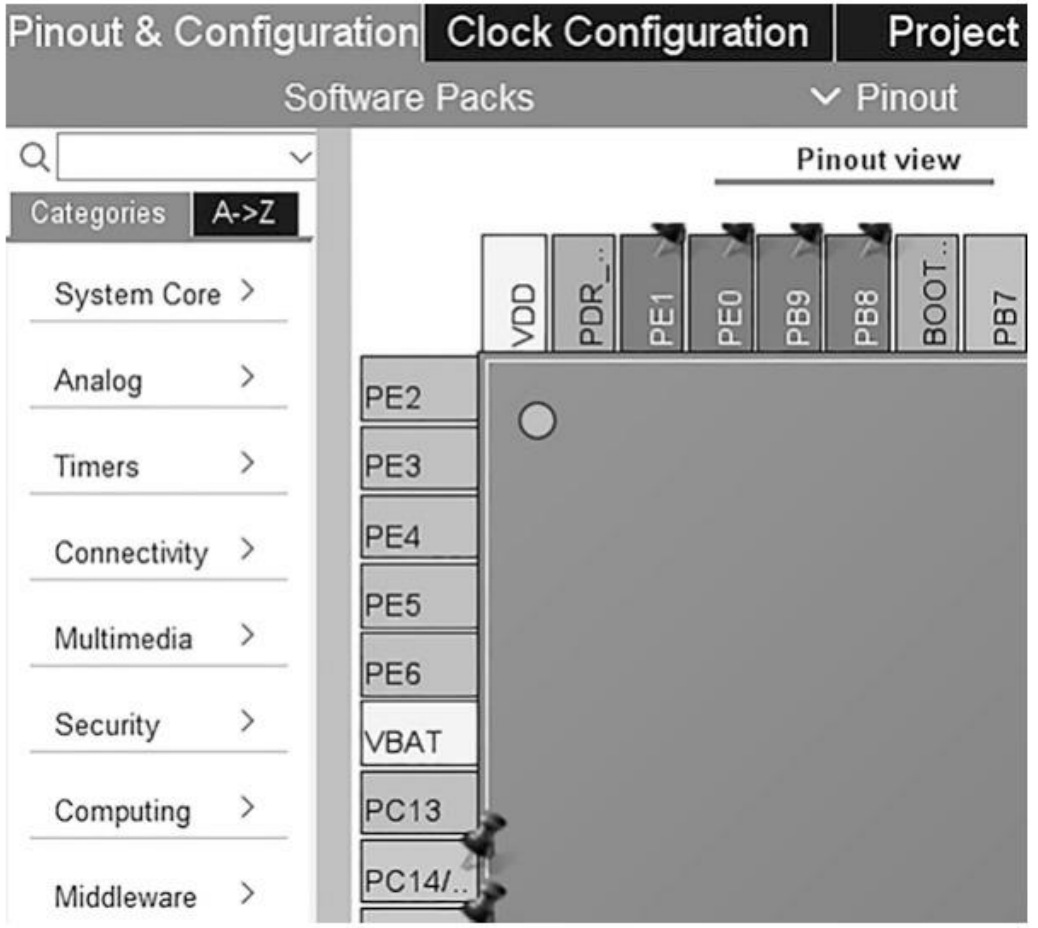


Fig. 3.7 Initial screen for STM32CubeMX

- To do so, we should first modify the microcontroller pin properties.
- Hence, we should set the pin PA0 of the STM32F4 microcontroller as “GPIO_Input” and pin PG13 as “GPIO_Output.”
- Fortunately, these modifications can be done in the STM32CubeMX interface easily.
- Within the "Pinout&Configuration” tab, locate the mentioned pins and left-click on them one by one.
- A pop-up window appears asking for which purpose the pin will be used for.
- Set the pin PA0 as “GPIO_Input” and pin PG13 as “GPIO_Output” and proceed to clock configuration by opening the “Clock Configuration” tab.
- We will benefit from the automatic clock setting property of STM32CubeMX here. To note here, this is the default setting and we suggest using this option whenever possible. Clock setup will be covered in detail in Chap. 6.

Generating and Modifying the Code

- Now, we are ready to generate the code corresponding to the hardware setup.
- Before doing so, we can open the “Project Manager” tab.
- Here, we can set the properties of our project from the “Project,” “Code Generator,” and “Advanced Settings” tabs.
- We suggest keeping all settings as they are since these are generated automatically.
- The next step is generating the code related to the hardware setup. To do so, press the “Save” button.
- A pop-up window appears asking for generating the code. Press “OK” there.

- As the code is generated, we can open the “main.c” file under the “Src” folder.
- As this file is opened, the reader will observe that there are sections labeled “USER CODE BEGIN” and “USER CODE END” asking for the user to add his or her code snippets.
- This setup is done to ensure that other parts of the code related to hardware setup are not modified by mistake.
- Finally, we will test the project. To do so, add the following C code snippet in Listing 3.2 to the appropriate place in the opened “main.c” source file.
- As we debug the project and embed the generated code on the STM32F4 microcontroller, it will be ready to be executed.
- The reader can check whether the code is running or not by pressing and releasing the user button on the STM32F4 board. Every time the button is pressed, the green LED on the board should turn on.

Listing 3.2 The C code snippet to be added

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0))
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
else
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Mbed and Mbed Studio Platforms

- Although STM32CubeIDE allows controlling all microcontroller properties, some users do not need such a detailed setup in their projects. Therefore, we introduce the web-based Mbed and its desktop version Mbed Studio in this section.
- Mbed on web has three main advantages.
- First, it does not require downloading and installing any IDE on PC. It is structured such that the user develops his or her project on a web-based platform.
- Second, Mbed is formed such that a code written for one platform in the Mbed ecosystem can easily be ported to another platform as long as hardware requirements are satisfied. This is possible since Mbed simplifies and generalizes low-level hardware setup and usage. From this perspective, Mbed resembles the popular Arduino platform.
- Third, Mbed allows code sharing between community members since it is an online platform. This is done by posting and importing the generated code through the web-based interface. In fact, the reader can easily use a project posted by another community member by importing it to his or her workspace. Then, this project can be developed further by the reader for his or her own needs.

Managing a Project in Mbed

- “main.cpp”
- We should replace the content of the file by the C++ code in Listing 3.3 for our purposes.

Listing 3.3 The main file for our project under Mbed

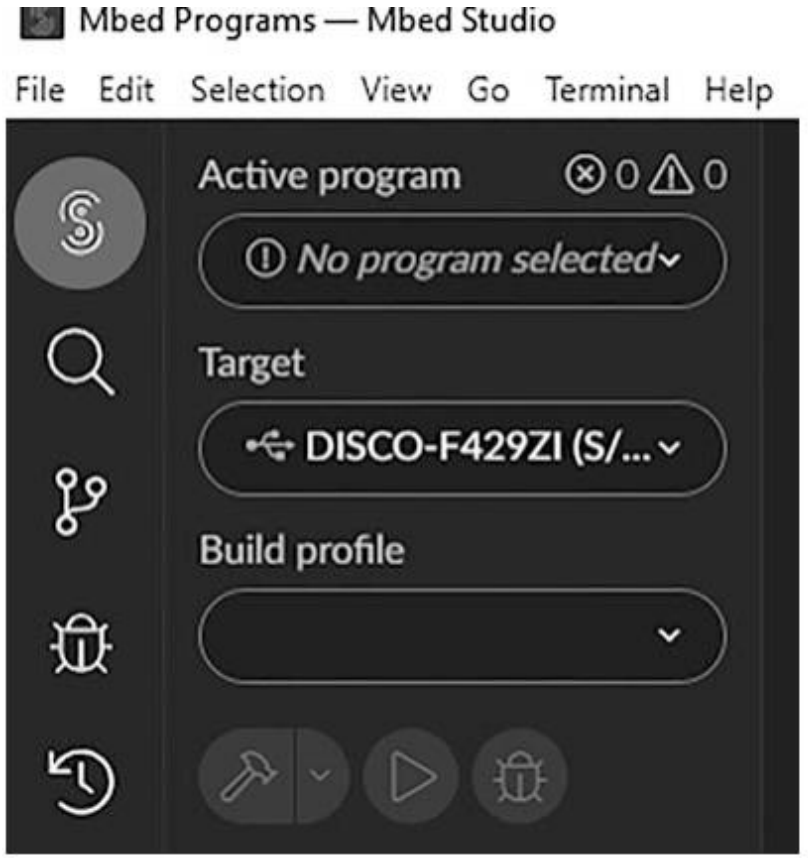
```
#include "mbed.h"

#define WAIT_TIME 500000
DigitalOut greenLED(LED1);

int main()
{
while (true)
{
greenLED = !greenLED;
wait_us(WAIT_TIME);
}
}
```

Mbed Studio on Desktop

- The procedure is straightforward in creating, building, and executing the project under Mbed online. However, the user should have an active Internet connection to perform all these operations. This may not be possible for some cases. Besides, it is not possible to debug the code during its execution. We can benefit from Mbed Studio to overcome these shortcomings.
- Mbed Studio is the desktop version of Mbed. The program will launch and the IDE will be as in Fig. 3.10. As can be seen in this figure, Mbed Studio has a fairly compact interface. There exists a panel on the left side of the window. Through it, the user can create a new project, select the target, and build the project for the selected target. At the center side of the window, the main code file will be opened as it is added to the project. There are tabs which will summarize all build and debug operations at the bottom of the panel.



Arduino IDE

- Arduino is an open-source electronics platform based on a simple hardware and software environment.
- Arduino boards can do what they do by programming the microcontroller on the board. The Arduino IDE is used for this, whichis open-source and is continually growing via a vast worldwide user-base.

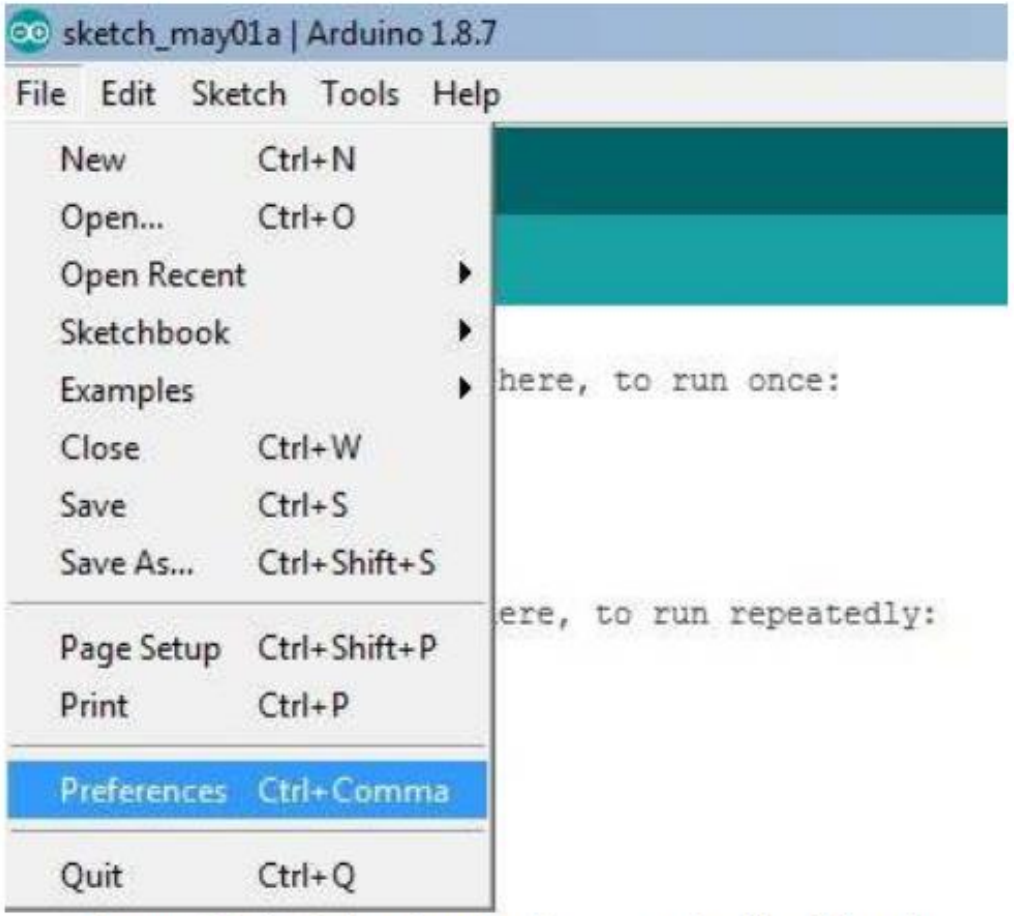


Figure 16.1: Selecting Preferences in the File tab

- In accordance with Figure 16.1 for installing the STM32duino library in the Arduino IDE environment, click the File tab and select preferences.
- In the preferences window, tick the compilation and output boxes (show verbose output during) to generate hex code for programming after successful compilation.
- Furthermore, in additional boards manager URL enter the following web link:

https://github.com/stm32duino/BoardManagerFiles/raw/master/STM32/package_stm_index.json

Click the OK button as shown in Figure 16.2. From the Tools tab and Board section, select the 'boards manage' icon as shown in Figure 16.3.

In the boards manager window, choose type as contributed as shown in Figure 16.4.

Select STM32 Cores by STMicroelectronics and install by pressing the install button as shown in Figure 16.5.

After installation, an INSTALLED message appears as shown in Figure 16.6.

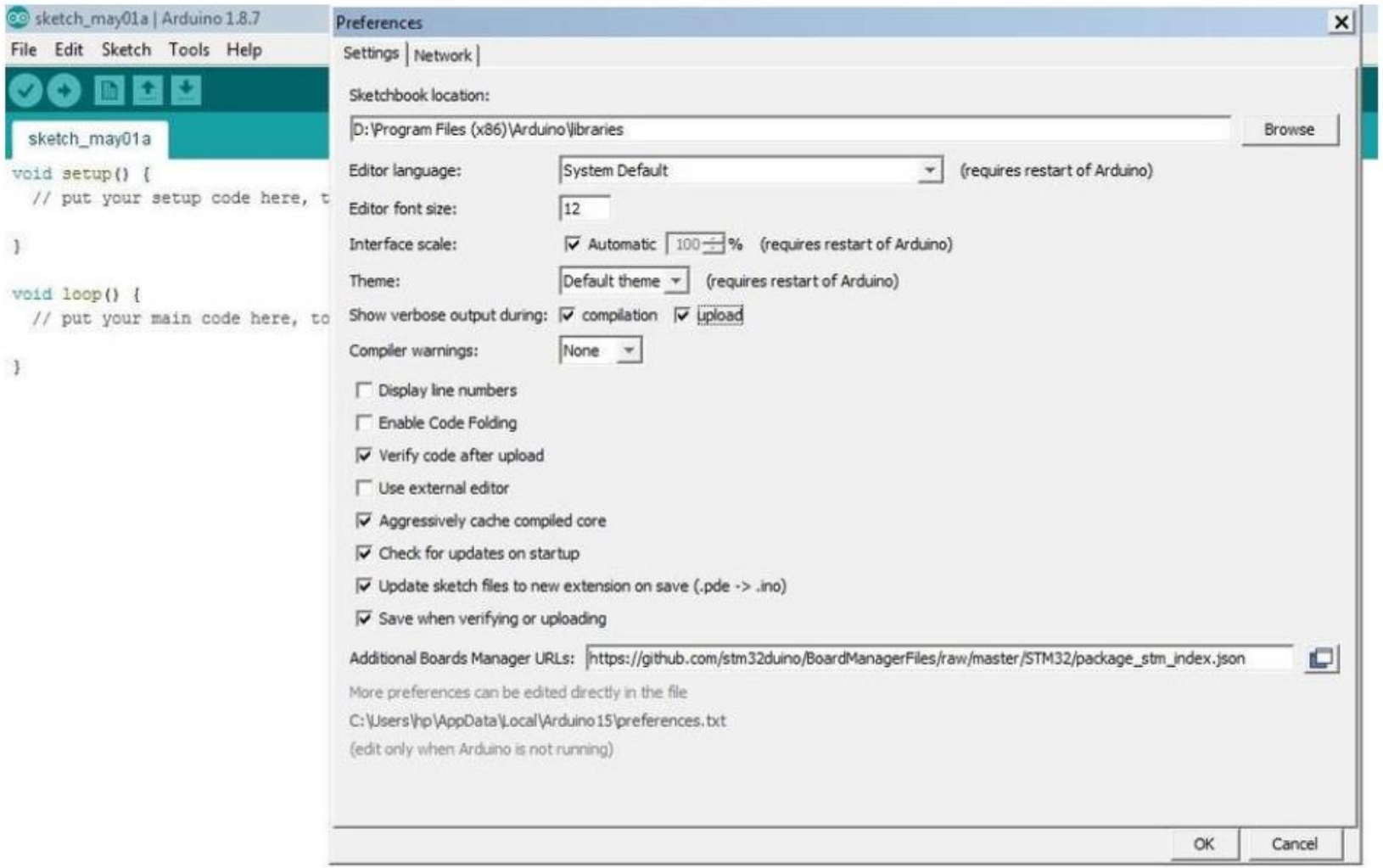


Figure 16.2: Preferences dialog settings

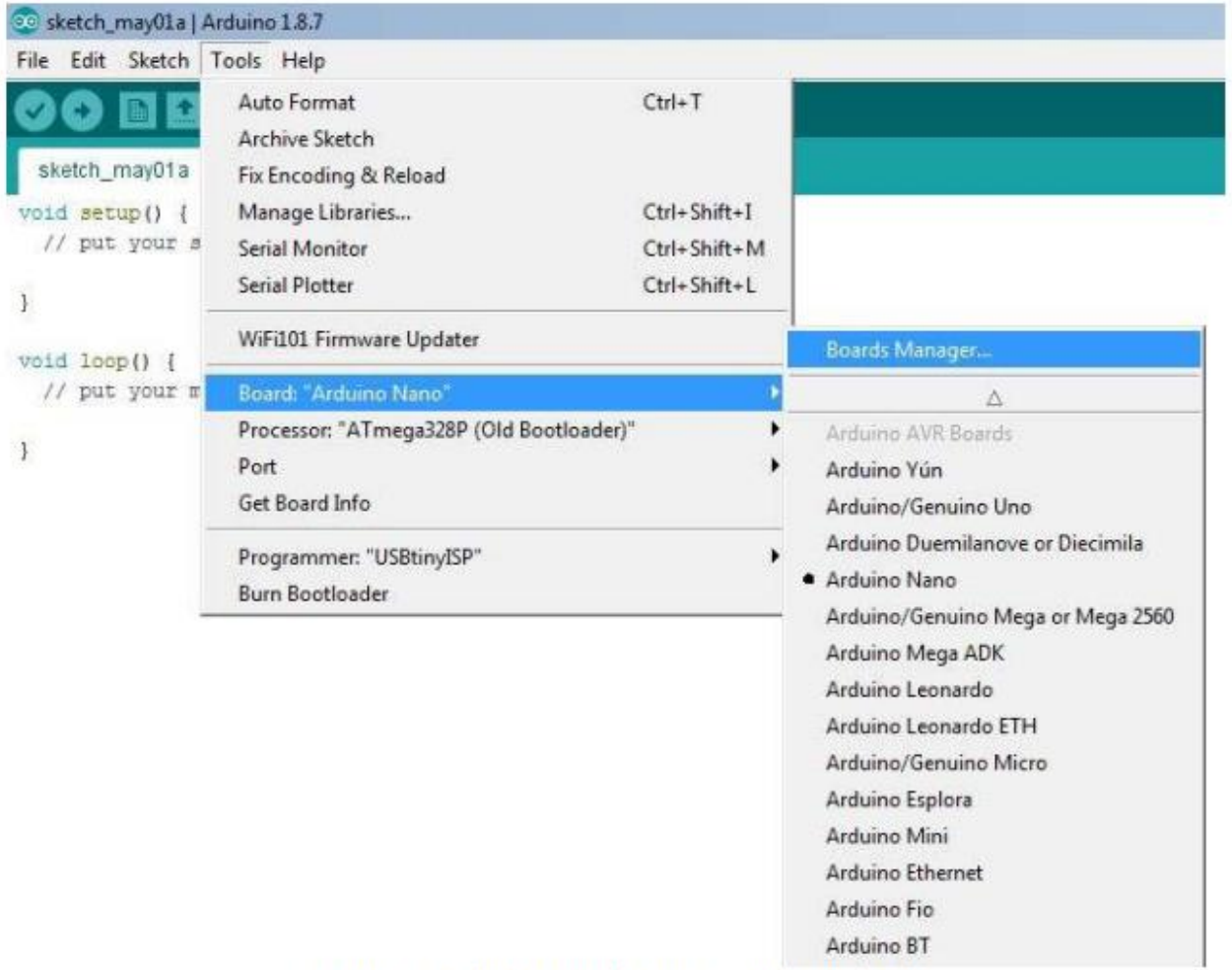


Figure 16.3: Selecting Boards Manager in the Tools tab

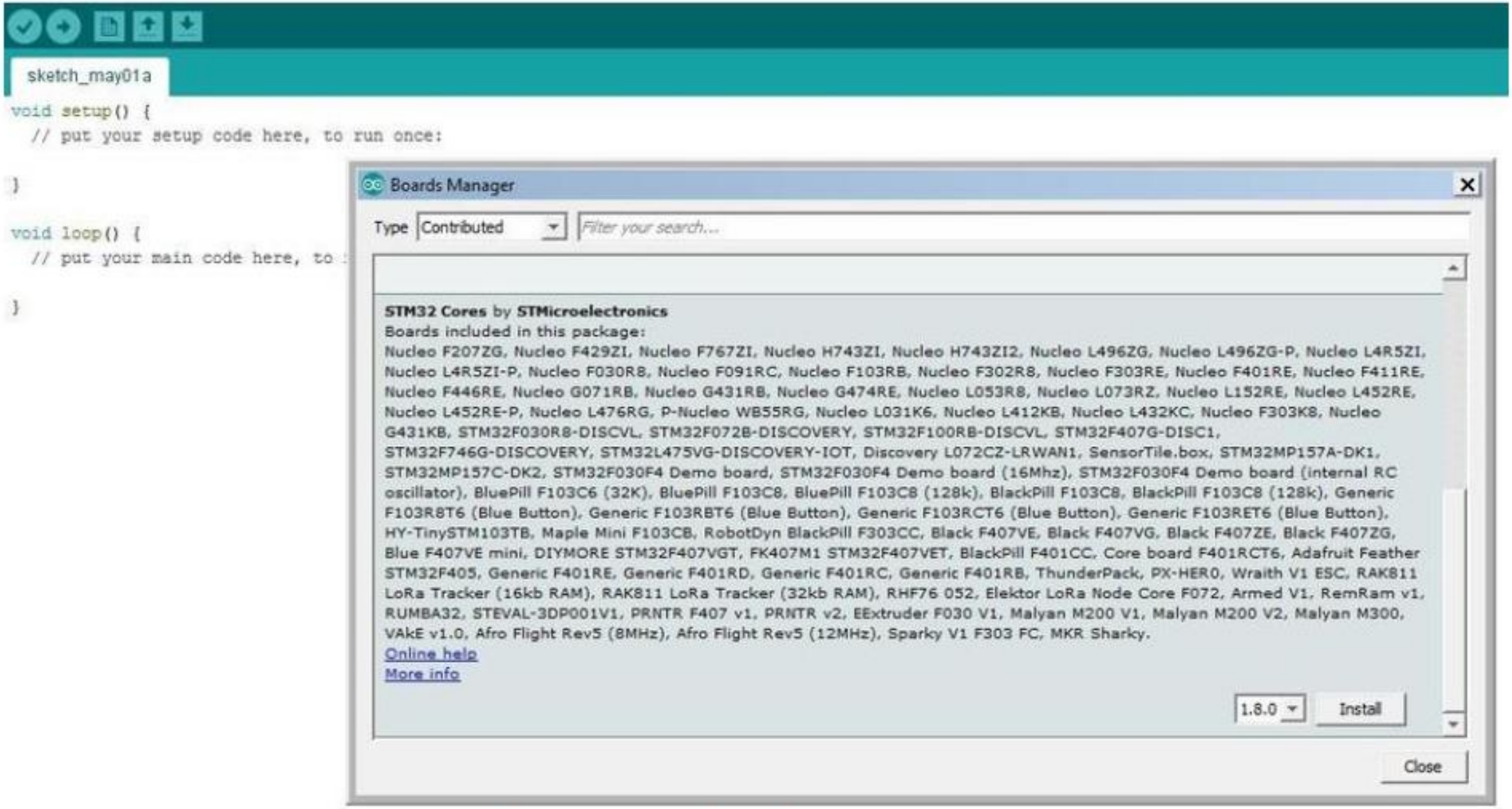


Figure 16.5: Installing STM32 Cores by STMicroelectronics

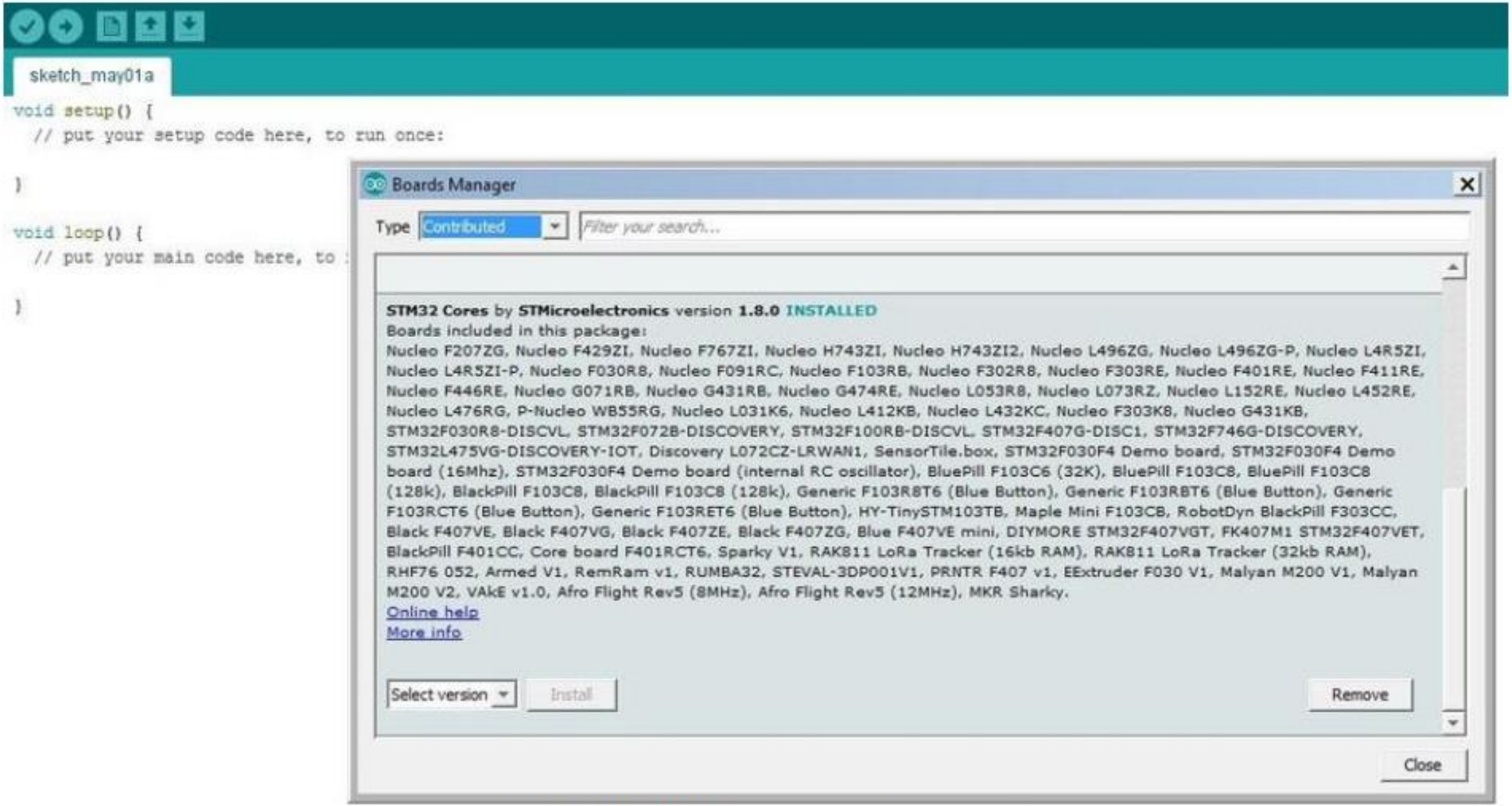


Figure 16.6: INSTALLED message after installation

- We can now check the installed STM32 boards by clicking on the Tools tab, and selecting the board and choosing, for example, the Generic STM32F1 series as shown in Figure 16.7.

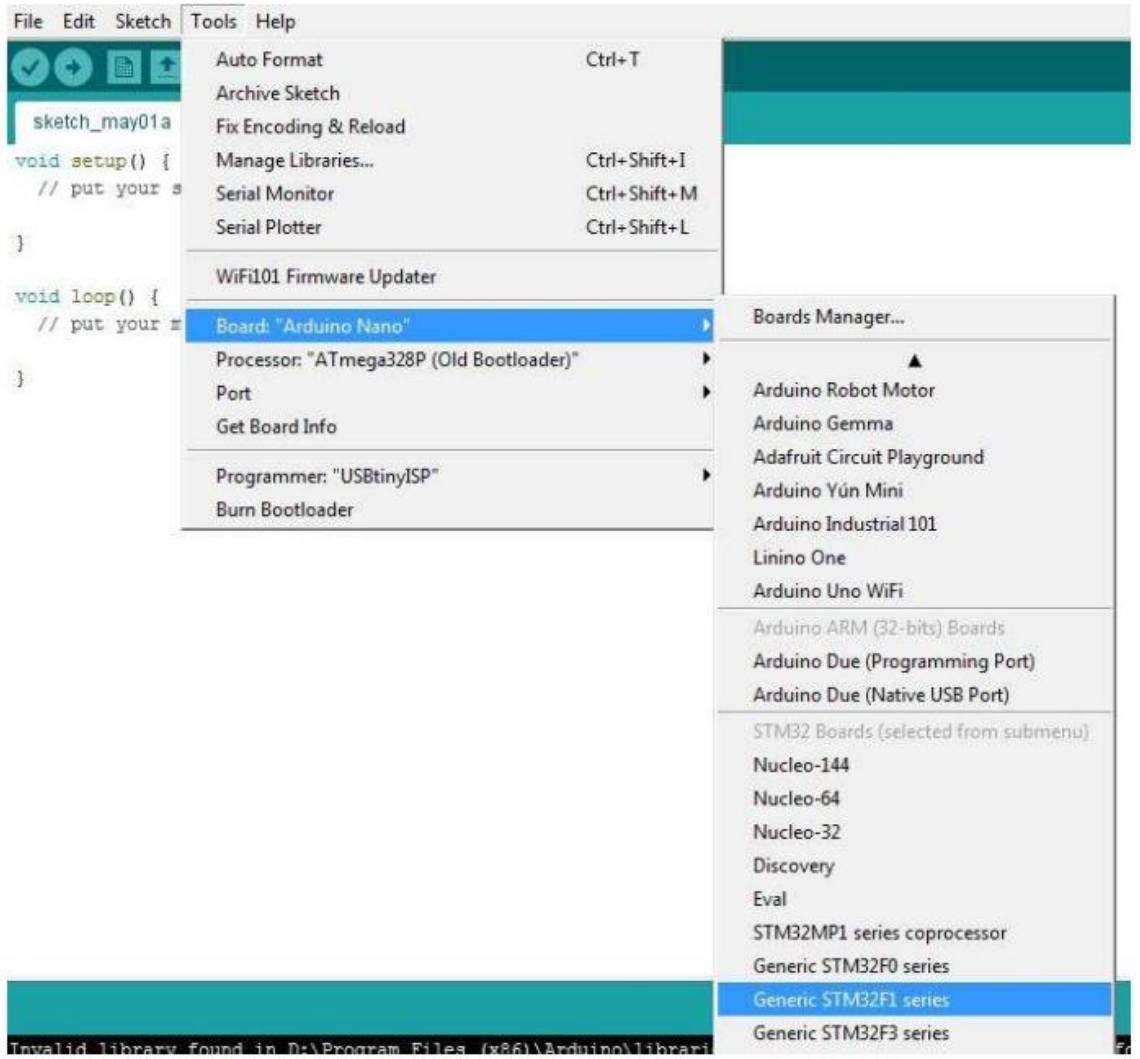


Figure 16.7: Choosing the Generic STM32F1 series from the board section

- If we click on the Tools tab again, we can see the Generic STM32F1 series board in the board part number section as illustrated in Figure 16.8.

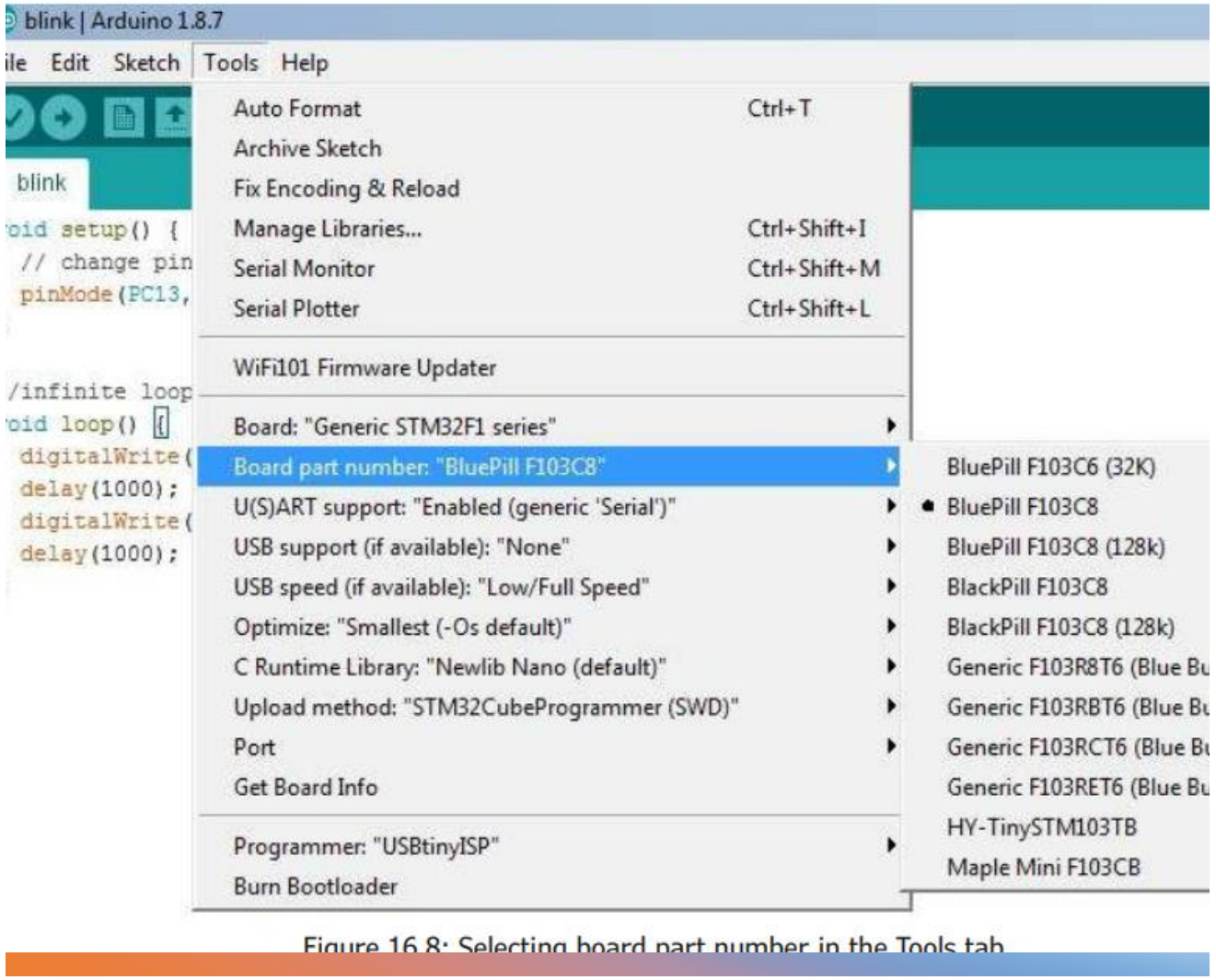


Figure 16.8: Selecting board part number in the Tools tab

- For flashing an LED connected to the PC13 pin of the STM32 blue pill board, the program is demonstrated in Figure 16.9.

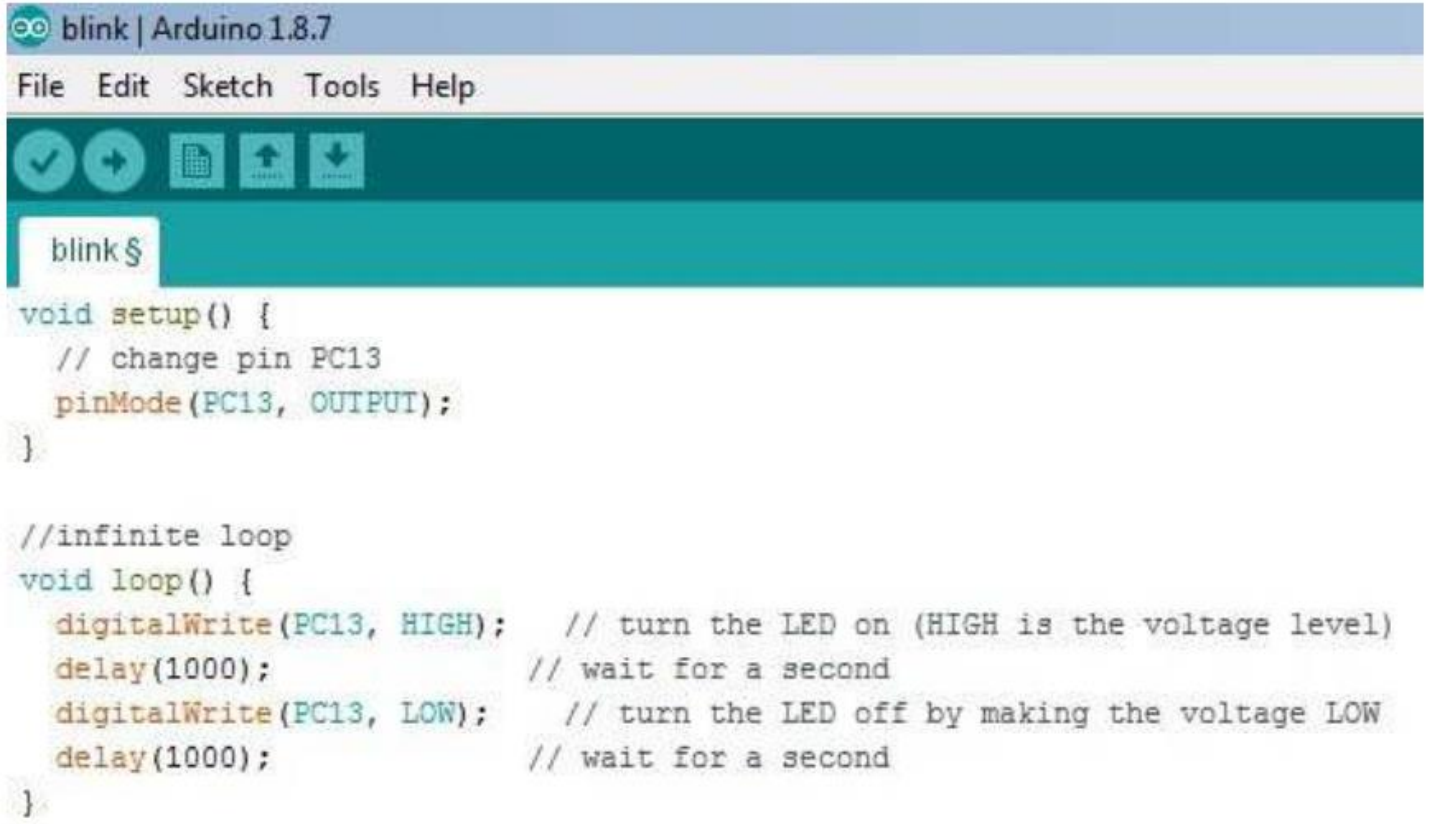


Figure 16.9: Flashing an LED program

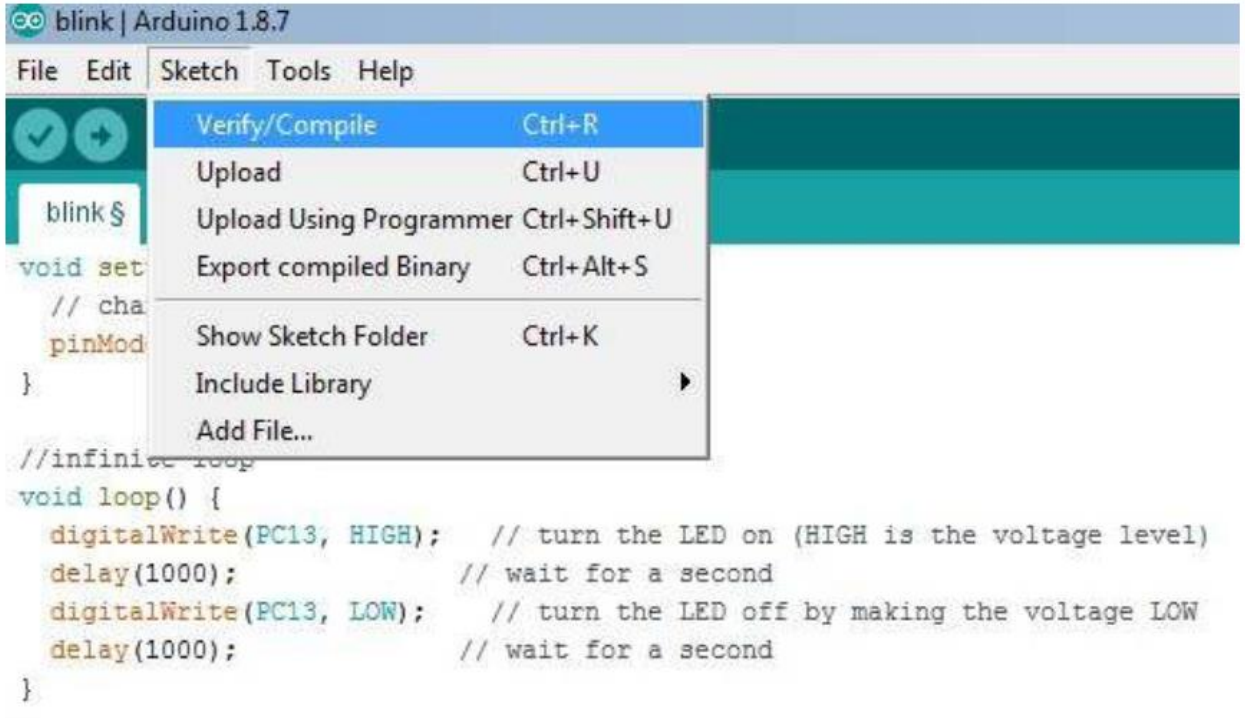


Figure 16.10: Compiling the program

- From the Sketch tab, select Verify/Compile to compile the project and generate the hex code for programming the microcontroller as shown in Figure 16.10.



Figure 16.11: Generated hex code accessing route message

- The hex code generated and its accessing route message are shown in Figure 16.11.

```
text
align 2
global main

ain:

equ     loc1,     0x20004000
equ     loc2,          0x30002000

emPrep:  LDR     R0, =loc1
        LDR     R1, =0xF0F1F2F3
        STR     R1, [R0]
        ADD     R0, R0, #4
        LDR     R1, =0xF4F5F6F7
        STR     R1, [R0]
        ADD     R0, R0, #4
        LDR     R1, =0xF8F9FAFB
        STR     R1, [R0]
        ADD     R0, R0, #4
        LDR     R1, =0xFCFDFEFF
        STR     R1, [R0]

        LDR     R3, =0x00//index
        LDR     R4, =0x00//jndex
        LDR     R0, =loc1//0x20004000
        MOV     R1, #0x1//mask to check odd
        MOV     R2, #0xff//mask to pick num
                MOV     R6, #0x0//summation
```

```
loop1:    LDR     R5, [R0]

loop2:   AND     R1, R1, R5
         CMP     R1, #1
         BNE     continue

         AND     R2, R2, R5
         ADD     R6, R6, R2

continue: LSR     R5, R5, #8
         MOV     R1, #0x1
         MOV     R2, #0xff
         ADD     R4, R4, #1
         CMP     R4, #4
         BLT     loop2

         LDR     R4, =0x00// reset jndex
         ADD     R0, R0, #4
         ADD     R3, R3, #1
         CMP     R3, #4
         BLT     loop1

         LSR     R6, R6, #3
         LDR     R0, =loc2
         STR     R6, [R0]

END: b END
 .end
```

Lab3 solution